ZombieCarCrash

ZombieCarCrash

장태훈

INDEX

목차

1 포트폴리오 개요

2 포트폴리오 설명

3 주요 구현 기술

01

포트폴리오 개요

게임 이름 : **ZombieCarCrash**

장르 : 아케이드 캐주얼 자동차 액션

개발환경 : Unity 2022.3.4f1 / C#

개발기간 : 10월31일 ~ 12월20일 (51일)

핵심 컨셉 : 좀비를 처치해 얻은 코인을 수집하고, 차량을 강화 하여 다음 스테이지로 진출하는 캐주얼 액션 게임

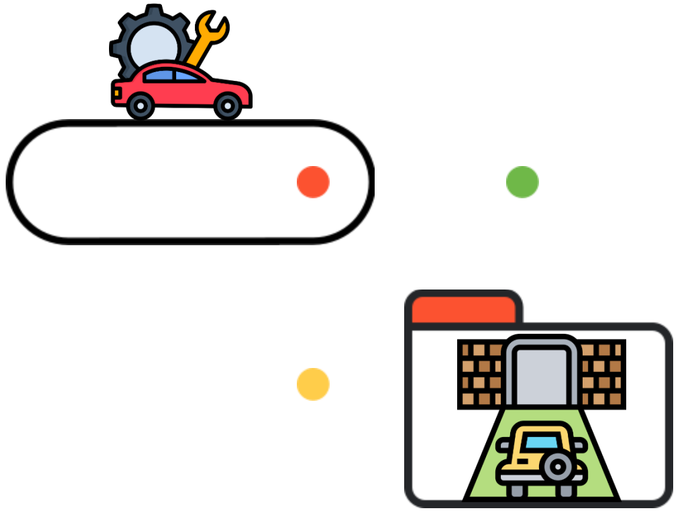
Github : https://github.com/BambaStone/ZombieCarCrash

빌드파일 : https://drive.google.com/file/d/1hkojFeR4ppI058niw0-hdfHiSjWj8IJB/view?usp=drive_link

플레이 영상 : https://youtu.be/VZOl3Ay0cPE

02

포트폴리오 설명



게임의 핵심 규칙

1

1단계 Farm

마을 밖 좀비 처치 및 코인 드랍

2

2단계 Collect

트럭 짐칸에 코인을 싣고(수집) 마을로 복귀

3

3단계 Upgrade

수집한 자원으로 차량의 성능 강화

4

4단계 Progress

강화된 차량으로 더 많은 코인을 모으고 모은 코인으로 다음 스테이지 해금

기본 키 조작

W : 전진
S :후진 및 브레이크
A : 좌회전
D : 우회전

Space : 사이드 브레이크

UI 설명 - 메인화면

Save1~3 버튼 :
클릭시 Save1~3의 데이터를 불러옵니다.

Quit 버튼 :
클릭시 게임을 종료합니다.

UI 설명 - 메인화면

세이브 데이터 창 :
Save1~3버튼으로 불러온 세이브 데이터를 표시하는 UI입니다.
항목별로
- Weapon – 현재 장착된 무기
- Coin – 현재 저장된 코인
- ZombieKill – 지금까지 잡은 좀비 수
- Damage~CoinTank – 현재 차량 강화 상태

를 의미합니다.

UI 설명 - 메인화면

Start 버튼 :
클릭시 불러온 세이브
데이터로 게임을 시작합니다.

Reset 버튼 :
클릭시 불러온 세이브
데이터를 지우고 해당
세이브칸 을 초기화 합니다.

UI 설명 - 게임화면

HP : 현재 차량의 HP를 표시한다.

CoinTank : 현재 차량에 실려있는 코인을 표시합니다.

Coin : 현재 저장된 코인을 표시합니다.

UI 설명 - 게임화면

Input Coin : 차량이 해당 구역에 들어가면 차량에 실린 코인을 저장합니다.

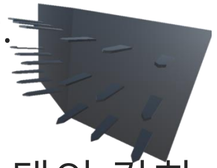
Car Upgrade : 차량이 해당 구역에 들어가면 차량 강화 UI를 오픈합니다.

UI 설명 - 게임화면

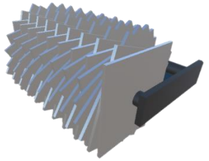
차량 강화 창 : 현재 차량의
강화 상태를 표시하고
각 스탯의 강화를 실행할수
있는 UI입니다.

UI 설명 - 게임화면

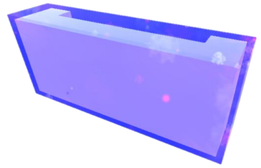
Weapon : 현재 차량에 장착된 무기를 표시하고, 교체할수 있다.
기본적으로 가시범퍼가 장착되어 있습니다.



아래의 Damage 스탯의 강화 정도에 따라
5레벨에는 원형톱날분쇄기가 해금되고,



10레벨에는 파워범퍼가 해금됩니다.



UI 설명 - 게임화면

Damage : 차량 무기의 데미지 강화 레벨을 표시하고, 버튼 클릭시 코인을 차감하고 강화합니다.

MaxSpeed : 차량의 최대 속도 강화 레벨을 표시하고, 버튼 클릭시 코인을 차감하고 강화합니다.

MaxAccele : 차량의 가속도 강화 레벨을 표시하고, 버튼 클릭시 코인을 차감하고 강화합니다.

UI 설명 - 게임화면

MaxHP : 차량의 최대HP 강화 레벨을 표시하고, 버튼 클릭시 코인을 차감하고 강화합니다.

HPRegen : 차량의 HP재생 강화 레벨을 표시하고, 버튼 클릭시 코인을 차감하고 강화합니다.

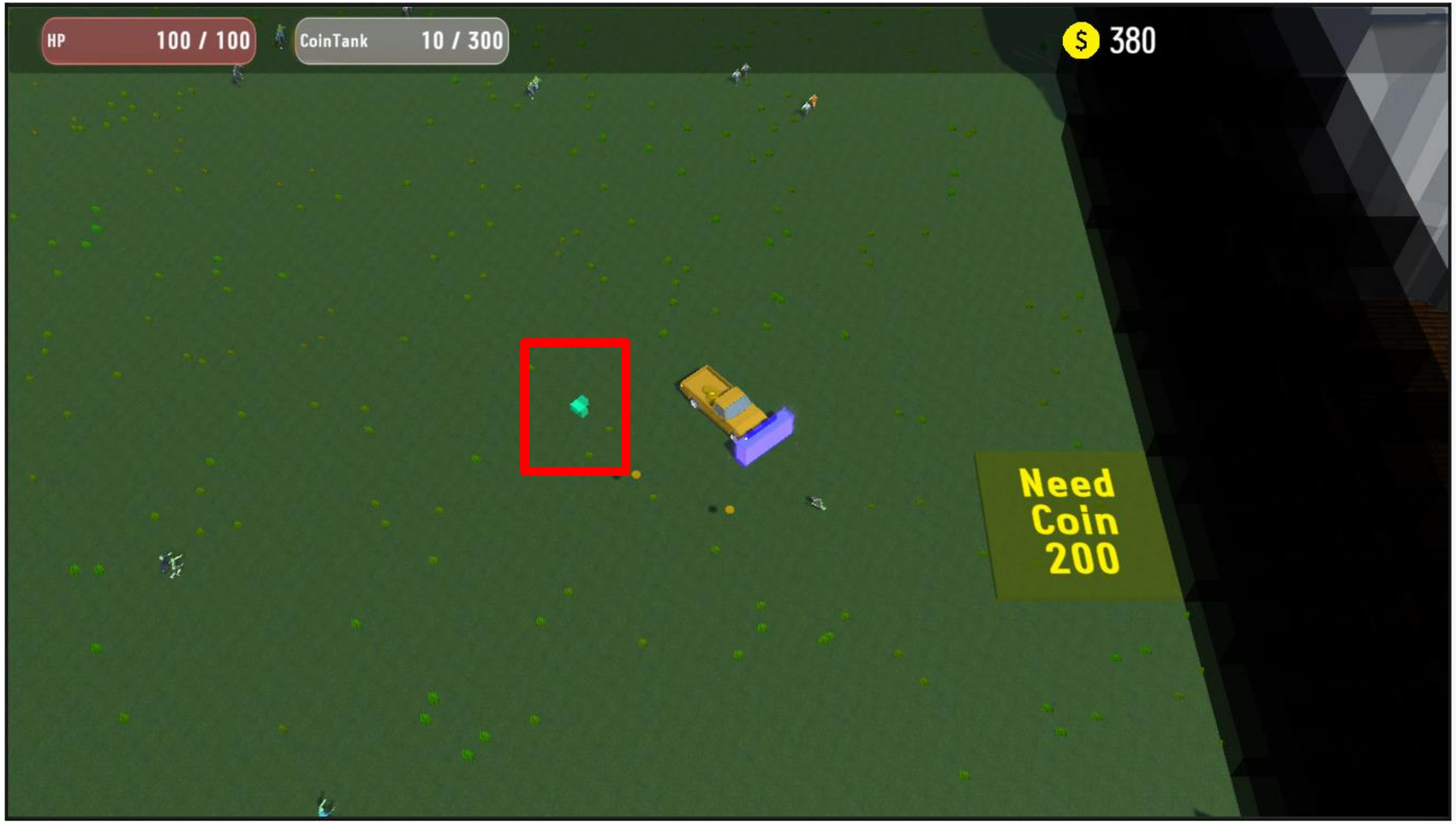
CoinTank : 차량의 실을수 있는 코인의 최대갯수 강화 레벨을 표시하고, 버튼을 클릭시 코인을 차감하고 강화합니다.

UI 설명 - 게임화면

Next Upgrade : 다음 강화에 필요한 코인 갯수를 표시해줍니다.
한번 강화 할때마다 1씩 늘어납니다.

UI 설명 - 게임화면

다음 스테이지 게이트 :
다음 스테이지로 가는 길을
막는 게이트입니다.
노란색구역(코인투입구)에
써있는 갯수만큼의 코인을
코인탱크에 실어서
노란색구역에 진입할시
실려있는 코인을 소모해서
다음 스테이지로 가는
게이트를 개방합니다.



UI 설명 - 게임화면

홈네비게이션 :
마을로 돌아가는 방향을
알려주는 네비게이션입니다.

03

주요 구현 기술

주요 구현 기술

- 휠 콜라이더를 이용한 차량 통제 시스템
- 코인의 드랍과 이동, 축적 시스템
- 싱글톤패턴을 이용한 차량의 업그레이드와 데이터 저장 시스템

휠 콜라이더를 이용한 차량 통제 시스템

- 바퀴가 돌아가는 모습을 구현하는 AnimateWheelMeshes()함수
- 가속을 구현한 Accele() 함수
- 핸드브레이크(사이드브레이크)를 구현한 HandBrake()함수와 UnHandBrake()함수
- 브레이크를 구현한 Brake()함수와 UnBrake()함수
- 차량의 회전(스티어링)을 구현한 TurnLeft()함수와 TurnRight()함수
- 차량이 회전중이 아닐때 스티어링을 복구하는 시스템을 구현한 ResetSteeringAngle()함수
- 드리프트 상태일 때 드리프트의 이펙트사용을 구현한 Drift()함수
- 위의 함수들을 이용하여 실시간으로 차량을 통제하는 Update()함수

SimpleCarController.cs

```
void AnimateWheelMeshes()
  {
    // 각 바퀴의 휠콜라이더에서 바퀴의 위치값과 회전값을 받아서
    // 바퀴의 매쉬에 적용
    Quaternion FLWRotation;
    Vector3 FLWPosition;                                          1
    WheelFLCollider.GetWorldPose(out FLWPosition, out FLWRotation);   2
    WheelFLMesh.transform.position = FLWPosition;
    WheelFLMesh.transform.rotation = FLWRotation;                 3

    Quaternion FRWRotation;
    Vector3 FRWPosition;
    WheelFRCollider.GetWorldPose(out FRWPosition, out FRWRotation);
    WheelFRMesh.transform.position = FRWPosition;
    WheelFRMesh.transform.rotation = FRWRotation;

    Quaternion RLWRotation;
    Vector3 RLWPosition;
    WheelBLCollider.GetWorldPose(out RLWPosition, out RLWRotation);   4
    WheelBLMesh.transform.position = RLWPosition;
    WheelBLMesh.transform.rotation = RLWRotation;

    Quaternion RRWRotation;
    Vector3 RRWPosition;
    WheelBRCollider.GetWorldPose(out RRWPosition, out RRWRotation);
    WheelBRMesh.transform.position = RRWPosition;
    WheelBRMesh.transform.rotation = RRWRotation;
  }
```


휠 콜라이더를 이용한 차량 통제 시스템

바퀴가 돌아가는 모습을 구현하는 AnimateWheelMeshes()함수

1. 바퀴의 휠콜라이더에서 회전값과 위치값을 받아올 변수를 선언합니다.
2. 휠콜라이더에서 회전 값과 위치 값을 받아옵니다.
3. 받아온 회전값과 위치값을 메쉬에 적용합니다.
4. 각 바퀴별로 같은 과정을 반복합니다.

SimpleCarController.cs

```
public void Accele()
  {
    _throttleAxis = _throttleAxis + (Time.deltaTime * 3f);  //부드럽게 스로틀 올리기
    if(_throttleAxis > 1f)  //스로틀이 1보다 커지면
    {
      _throttleAxis = 1f; //1로고정
    }
    //소수점을 반올림한 현재 차의 속도가 최고속도에 도달하지 못했을때
    if (Mathf.RoundToInt(CarSpeed) < MaxSpeed)
    {
      //모든 바퀴의 모터 토크에 차의 가속도와 스로틀상태에 따른 양의 수를 적용한다.
      WheelFLCollider.motorTorque = (Acceleration * 50f) * _throttleAxis;
      WheelFRCollider.motorTorque = (Acceleration * 50f) * _throttleAxis;
      if (!UseHandBrake)//핸드브레이크 일때는 뒷바퀴는 제외
      {
        WheelBLCollider.motorTorque = (Acceleration * 50f) * _throttleAxis;
        WheelBRCollider.motorTorque = (Acceleration * 50f) * _throttleAxis;
      }
    }
    else   //소수점을 반올림한 현재 차의 속도가 최고속도에 다다르면
    {
      //모든 바퀴의 모터토크에 0을 적용한다.
      WheelFLCollider.motorTorque = 0;
      WheelFRCollider.motorTorque = 0;
      WheelBLCollider.motorTorque = 0;
      WheelBRCollider.motorTorque = 0;
    }
  }
```

1

2

휠 콜라이더를 이용한 차량 통제 시스템

가속을 구현한 Accele() 함수

1. 현재 스로틀 상태를 변경합니다.

   _throttleAxis(현재 스로틀 상태)를 초당 3의 속도로 1까지 부드럽게 올리고, 1이되면 고정합니다.

2. CarSpeed(현재속도)가 MaxSpeed(최대속도)가 될때까지 차의 가속을 변경합니다.

   휠콜라이더의 motorTorque는 바퀴의 RPM에 양수면 앞으로 음수면 뒤로 회전값을 더하게 됩니다.

   따라서 MaxSpeed에 도달하기 전까진 motorTorque에 지속적으로 Acceleration(차의 가속도)에 기본값50과 _throttleAxis를 곱한값을 적용합니다.

   MaxSpeed에 도달하면 더이상의 추가 가속이 없게 motorTorque를 0으로 바꿔줍니다.

SimpleCarController.cs

```
public void HandBrake()
{
    //모든 뒷바퀴의 모터토크에 0을 적용한다.
    WheelBLCollider.motorTorque = 0;
    WheelBRCollider.motorTorque = 0;
    //모든 뒷바퀴의 브레이크토크에 브레이크파워를 적용
    WheelBLCollider.brakeTorque = BreakPower;
    WheelBRCollider.brakeTorque = BreakPower;
    //뒷바퀴의 접지력 약화
    _wheelBLFriction.extremumSlip = _wheelBLExtremumSlip*5f;
    WheelBLCollider.sidewaysFriction = _wheelBLFriction;
    _wheelBRFriction.extremumSlip = _wheelBRExtremumSlip*5f;
    WheelBRCollider.sidewaysFriction = _wheelBRFriction;
}
public void UnHandBrake()
{
    //모든 뒷바퀴의 브레이크토크에 0 적용
    WheelBLCollider.brakeTorque = 0;
    WheelBRCollider.brakeTorque = 0;
    //뒷바퀴의 접지력 복구
    _wheelBLFriction.extremumSlip = _wheelBLExtremumSlip;
    WheelBLCollider.sidewaysFriction = _wheelBLFriction;
    _wheelBRFriction.extremumSlip = _wheelBRExtremumSlip;
    WheelBRCollider.sidewaysFriction = _wheelBRFriction;
}
```

1

2

3

1

2

휠 콜라이더를 이용한 차량 통제 시스템

핸드브레이크(사이드브레이크)를 구현한 HandBrake()함수와 UnHandBrake()함수

HandBrake()

1. 뒷바퀴의 모터토크에 0을 적용해서 더이상 RPM이 늘어나지 않게합니다.
2. 휠콜라이더의 brakeTorque는 바퀴의 RPM을 수치만큼 현재방향의 반대방향으로 적용시킵니다. 따라서 뒷바퀴의 브레이크토크에 높은값을 입력해서 RPM을 순식간에 줄어들게 합니다.
3. 휠콜라이더의 sidewaysFriction은 바퀴의 종방향 접지력을 의미하고, 그 안의 extremumSlip은 최대 미끄러짐 수치를 의미합니다. 따라서 extremumSlip의 값을 올려서 옆으로 미끄러지는 드리프트를 구현할수 있게 합니다.

UnHandBrake()

1. 브레이크토크에 0을 적용해서 더이상 RPM이 줄어들지 않게 합니다.
2. extremumSlip값을 되돌려서 다시 옆으로 미끄러지지 않는 정상적인 주행이 되도록 합니다.

SimpleCarController.cs

```
public void Brake()
{
    //모든 바퀴의 모터토크에 0을 적용한다.
    WheelFLCollider.motorTorque = 0;
    WheelFRCollider.motorTorque = 0;
    WheelBLCollider.motorTorque = 0;
    WheelBRCollider.motorTorque = 0;
    //모든 바퀴의 브레이크토크에 브레이크파워를 적용
    WheelFLCollider.brakeTorque = BreakPower;
    WheelFRCollider.brakeTorque = BreakPower;
    WheelBLCollider.brakeTorque = BreakPower;
    WheelBRCollider.brakeTorque = BreakPower;
}
public void UnBrake()
{
    //모든 바퀴의 브레이크토크에 0을 적용
    WheelFLCollider.brakeTorque = 0;
    WheelFRCollider.brakeTorque = 0;
    WheelBLCollider.brakeTorque = 0;
    WheelBRCollider.brakeTorque = 0;
}
```

1

2

1

휠 콜라이더를 이용한 차량 통제 시스템

브레이크를 구현한 Brake()함수와 UnBrake()함수

Brake()

1. 모든바퀴의 모터토크에 0을 적용해서 더이상 RPM이 늘어나지 않게합니다.
2. 모든 바퀴의 브레이크토크에 높은값을 입력해서 RPM을 순식간에 줄어들게 합니다.

UnBrake()

1. 브레이크토크에 0을 적용해서 더이상 RPM이 줄어들지 않게 합니다.

SimpleCarController.cs

```
public void TurnLeft()
  {
    //스티어링 상태를 스티어링 속도에 따라 부드럽게 내리기
    _steeringAxis = _steeringAxis - (Time.deltaTime * 10f * SteeringSpeed);
    if (_steeringAxis < -1f)//스티어링 상태가 -1보다 작아지면
    {
      _steeringAxis = -1f;//스티어링 상태를 -1로 고정
    }
    //스티어링 상태 * 최대 스티어링 각도 = 목표 스티어링 각도
    float steering = _steeringAxis * MaxSteering;
    //현재 각 앞바퀴의 스티어링 각도를 목표 스티어링 각도를 향해 스티어링 속도에 따라 선형 보간을 통해 부드럽게 회전
    WheelFLCollider.steerAngle = Mathf.Lerp(WheelFLCollider.steerAngle, steering, SteeringSpeed);
    WheelFRCollider.steerAngle = Mathf.Lerp(WheelFLCollider.steerAngle, steering, SteeringSpeed);
  }

  public void TurnRight()
  {
    //스티어링 상태를 스티어링 속도에 따라 부드럽게 올리기
    _steeringAxis = _steeringAxis + (Time.deltaTime * 10f * SteeringSpeed);
    if (_steeringAxis > 1f)//스티어링 상태가 1보다 커지면
    {
      _steeringAxis = 1f;//스티어링 상태를 1로 고정
    }
    //스티어링 상태 * 최대 스티어링 각도 = 목표 스티어링 각도
    float steering =  steeringAxis * MaxSteering;
    //현재 각 앞바퀴의 스티어링 각도를 목표 스티어링 각도를 향해 스티어링 속도에 따라 선형 보간을 통해 부드럽게 회전
    WheelFLCollider.steerAngle = Mathf.Lerp(WheelFLCollider.steerAngle, steering, SteeringSpeed);
    WheelFRCollider.steerAngle = Mathf.Lerp(WheelFLCollider.steerAngle, steering, SteeringSpeed);
  }
```

1

2

3

1

2

3

휠 콜라이더를 이용한 차량 통제 시스템

차량의 회전(스티어링)을 구현한 TurnLeft()함수와 TurnRight()함수

1. 현재 스티어링상태를 정의하는 코드입니다.
   _steeringAxis 를 초당 10*SteeringSpeed(스티어링속도)의 속도로 1(우회전)또는 -1(좌회전)까지 부드럽게 올리거나 내리고, 각 1또는 -1이되면 고정합니다.

2. MaxSteering(최대 스티어링 각도)에 SteeringAxis(현재 스티어링 상태)를 곱해서 목표 스티어링 각도를 선언해줍니다.

3. 휠콜라이더의 steerAngle는 바퀴의 스티어링을 정의하며 양수면 오른쪽으로 음수면 왼쪽으로 회전하게 합니다. 그에 맞게 현재 앞바퀴의 스티어링 각도를 목표스티어링 각도를 향해 스티어링 속도에 따라 선형 보간을 통해 부드럽게 회전시킵니다.

SimpleCarController.cs

```
public void ResetSteeringAngle()
  {
      if (_steeringAxis < 0f) //스티어링 상태가 0보다 작을때
      {
         _steeringAxis = _steeringAxis + (Time.deltaTime * 10f *
SteeringSpeed);//스티어링상태를 부드럽게 올리기
      }
      else if (_steeringAxis > 0f)//스티어링 상태가 0보다 클때
      {
         _steeringAxis = _steeringAxis - (Time.deltaTime * 10f *
SteeringSpeed);//스티어링상태를 부드럽게 내리기
      }
      if (Mathf.Abs(WheelFLCollider.steerAngle) < 1f)//바퀴의 스티어링 각도가 1f보다 작을때
      {
         _steeringAxis = 0f;//스티어링 상태 0적용
      }

      //목표 스티어링 각도 = 스티어링 상태 * 최대 스티어링 각도
      float steeringAngle = _steeringAxis * MaxSteering;

      //현재 각 앞바퀴의 스티어링 각도를 목표 스티어링 각도를 향해 스티어링 속도에
따라 선형 보간을 통해 부드럽게 회전
      WheelFLCollider.steerAngle = Mathf.Lerp(WheelFLCollider.steerAngle, steeringAngle,
SteeringSpeed);
      WheelFRCollider.steerAngle = Mathf.Lerp(WheelFRCollider.steerAngle, steeringAngle,
SteeringSpeed);
  }
```

1

2

3

휠 콜라이더를 이용한 차량 통제 시스템

차량이 회전중이 아닐때 스티어링을 복구하는 시스템을 구현한 ResetSteeringAngle()함수

1. 현재 스티어링상태를 변경하는 코드입니다.
   _steeringAxis가 0이 아닐때 초당 10*SteeringSpeed(스티어링속도)의 속도로 0까지 부드럽게 올리거나 내립니다.

2. steerAngle(콜라이더의 스티어링 회전값)의 절댓값이 1보다 작아지면 _steeringAxis를 0으로 만듭니다.

3. _steeringAxis(현재스티어링 상태)에 맞춰서 steerAngle을 부드럽게 조정합니다.

SimpleCarController.cs

```
public void Drift()
  {
    if (EffectsUse)                    //이펙트를 사용하기로 했을 때
    {
      if (Drifting)                    //드리프트 중일때
      {
        //현재 X축 로컬 속도가 2.5f보다 높거나, 브레이크상태일때
        if (Mathf.Abs(_localVelocityX) > 2.5f || UseBrake)
        {
          if (WheelFLMist != null && WheelFRMist != null) //오른쪽과 왼쪽 앞바퀴 모두
연기 이펙트가 있을시
          {
            WheelFLMist.Play(); //연기 이펙트 실행
            WheelFRMist.Play();
          }
          if (WheelFLSkid != null && WheelFRSkid != null) //오른쪽과 왼쪽 앞바퀴 모두
스키드 마크 이펙트가 있을때
          {
            WheelFRSkid.emitting = true;   //스키드마크 그리기
            WheelFLSkid.emitting = true;
          }
        }
        if (WheelBLMist != null && WheelBRMist != null) //오른쪽과 왼쪽 뒷바퀴 모두 연기
이펙트가 있을시
        {
          WheelBLMist.Play(); //연기 이펙트 실행
          WheelBRMist.Play();
        }
        if (WheelBLSkid != null && WheelBRSkid != null) //오른쪽과 왼쪽 뒷바퀴 모두
스키드 마크 이펙트가 있을때
        {
          WheelBRSkid.emitting = true;   //스키드마크 그리기
          WheelBLSkid.emitting = true;
        }
      }
```

1

2

휠 콜라이더를 이용한 차량 통제 시스템

드리프트 상태일 때 드리프트의 이펙트사용을 구현한 Drift()함수 1/2

1. 드리프트 이펙트 사용여부와 현재 드리프트 상태 여부를 판별합니다

2. 드리프트 이펙트를 사용하면서 드리프트 상태일때

   a. 만약 X축 로컬 속도가 2.5보다 높거나(차량이 옆으로 미끄러지는 중), 브레이크 상태일때(차량이 움직이다가 갑자기 멈출때) 차량의 앞바퀴의 연기이펙트와 스키드마크 이펙트를 실행합니다.

   b. 뒷바퀴의 연기와 스키드마크도 실행해줍니다.

SimpleCarController.cs

```
        else //드리프트중이 아닐때
        {
            if (WheelFLMist != null && WheelFRMist != null)//오른쪽과 왼쪽 앞바퀴 모두 연기
이펙트가 있을시
            {
                WheelFLMist.Stop(); //연기 이펙트 멈춤
                WheelFRMist.Stop();
            }
            if (WheelFLSkid != null && WheelFRSkid != null) //오른쪽과 왼쪽 앞바퀴 모두 스키드
마크 이펙트가 있을때
            {
                WheelFRSkid.emitting = false;   //스키드마크 그리지않기
                WheelFLSkid.emitting = false;
            }
            if (WheelBLMist != null && WheelBRMist != null) //오른쪽과 왼쪽 뒷바퀴 모두 연기
이펙트가 있을시
            {
                WheelBLMist.Stop(); //연기 이펙트 멈춤
                WheelBRMist.Stop();
            }
            if (WheelBLSkid != null && WheelBRSkid != null) //오른쪽과 왼쪽 뒷바퀴 모두
스키드 마크 이펙트가 있을때
            {
                WheelBRSkid.emitting = false;   //스키드마크 그리지않기
                WheelBLSkid.emitting = false;
            }
        }
    }
  }
```

1

휠 콜라이더를 이용한 차량 통제 시스템

드리프트 상태일 때 드리프트의 이펙트사용을 구현한 Drift()함수 2/2

1. 만약 드리프트 중이 아닐 때 에는 모든 바퀴의 연기이펙트와 스키드마크 이펙트의 실행을 중지합니다.

SimpleCarController.cs

```
void Update()
  {
    //현재속도 저장 ( 1 m/s = 3.6 km/h )
    _localVelocity = _carRigidbody.velocity.magnitude;
    CarSpeed = _localVelocity * 3.6f;
    //현재 x의 로컬 속도를 저장 -> 현재 드리프트 중인지 확인하는데 사용
    _localVelocityX = transform.InverseTransformDirection(_carRigidbody.velocity).x;
    //현재 z의로컬 속도를 저장 -> 현재 자동차가 전진중인지 후진중인지 확인하는데 사용
    _localVelocityZ = transform.InverseTransformDirection(_carRigidbody.velocity).z;

    /*
     전진 가속 활성화 조건
       브레이크 상태가 아닐것(핸드브레이크는 괜찮음)
       후진가속상태가 아닐것 ( 만약 후진가속상태일 경우 브레이크 후에 가속상태로 변경 )
     후진 가속 활성화 조건
       브레이크 상태가 아닐것(핸드브레이크는 괜찮음)
       전진가속상태가 아닐것 ( 만약 전진가속상태일 경우 브레이크 후에 가속상태로 변경 )
     감속 활성화 조건
       브레이크,핸드브레이크,가속,후진상태가 아닐때 속도가 0이 아니라면
     브레이크 활성화 비활성화 조건
       전진중 후진 활성화,후진중 전진 활성화
       바퀴의 RPM이 0이되면 비활성화
     */
    if(WheelFLCollider.rpm==0)  //바퀴의 RPM이 0일때 (바퀴가멈춘상태)
    {
       UseBrake = false;      //브레이크상태 해제
       UnBrake();           //브레이크토크 원상복귀
    }
```

1

2

3

3. 주요 구현 기술

휠 콜라이더를 이용한 차량 통제 시스템

위의 함수들을 이용하여 실시간으로 차량을 통제하는 Update()함수 1/4

1. 현재 상태 파악을 위한 변수를 선언해줍니다.

   CarSpeed : 현재 속도

   _localVelocityX = 차량의 x축 속도, 드리프트 상태 체크

   _localVelocityZ = 차량의 z축 속도, 현재 차량의 전,후진 상태 체크

2. 가속과 브레이크 활성화 조건을 정리한 주석입니다.

3. 브레이크의 비활성화 조건에 따라서 RPM이 0이 될 때 브레이크를 해제합니다.

SimpleCarController.cs

1

```
    if(Input.GetKey(KeyCode.W))//W키를 누르고 있을때
    {
      if (UseBackAccele)        //후진이 작동중이면
      {
        UseBackAccele = false;  //후진상태 해제
      }
      if (WheelFLCollider.rpm < 0) //RPM이 0보다 작으면(바퀴가 뒤로돌때)
      {
        UseBrake = true;       //브레이크 상태 작동
      }
      if(!UseBrake && !UseAccele && !UseBackAccele) //브레이크,가속,후진 전부 작동중이
아닐때
      {
        UseAccele = true;              //가속 상태 작동
      }
    }
    if(!Input.GetKey(KeyCode.W))          //W키를 안누르고있을때
    {
      UseAccele = false;              //가속상태 해제
    }
```

2

휠 콜라이더를 이용한 차량 통제 시스템

위의 함수들을 이용하여 실시간으로 차량을 통제하는 Update()함수 2/4

1. 전진 가속 활성화 조건에 따라서 W키를 누르고 있을때
   a. 후진이 작동 중 이면 우선 후진을 해제합니다.
   b. 만약 바퀴의 RPM이 0보다 작으면 (바퀴가 뒤로 돌고있으면) 브레이크를 작동합니다.
   c. 브레이크, 전진 가속, 후진 가속 전부 작동중이 아니라면 전진 가속을 작동합니다.

2. W키를 누르고 있지 않다면 전진 가속을 해제합니다.

SimpleCarController.cs

```
    if (Input.GetKey(KeyCode.S))//S키를 누르고 있을때
    {
        if (UseAccele)              //가속이 작동중이면
        {
            UseAccele = false;      //가속상태 해제
        }
        if (_localVelocityZ > 0)     //RPM이 0보다 크면(바퀴가 앞으로돌때)
        {
            UseBrake = true;        //브레이크상태  작동
        }
        if (!UseBrake && !UseAccele && !UseBackAccele) //브레이크,가속,후진 전부 작동중이
아닐때
        {
            UseBackAccele = true;               //후진상태 작동
        }
    }
    if (!Input.GetKey(KeyCode.S))          //S키를 땟을때
    {
        UseBackAccele = false;              //후진상태 해제
    }
```

1

2

휠 콜라이더를 이용한 차량 통제 시스템

위의 함수들을 이용하여 실시간으로 차량을 통제하는 Update()함수 3/4

1. 후진 가속 활성화 조건에 따라서 S키를 누르고 있을때
    a. 전진 가속 이 작동 중 이면 우선 전진을 해제합니다.
    b. 만약 바퀴의 RPM이 0보다 크면 (바퀴가 앞으로 돌고있으면) 브레이크를 작동합니다.
    c. 브레이크, 전진 가속, 후진 가속 전부 작동중이 아니라면 후진 가속을 작동합니다.

2. S키를 누르고 있지 않다면 후진 가속을 해제합니다.

SimpleCarController.cs

```
    //현재 x축 로컬 속도가 2.5f보다 높거나, 브레이크또는 핸드브레이크 상태면서 차의 속력이 0이 아닐때 또는 전진상태인데 z축 로컬속도가 음수이거나 후진상태일때 로컬속도가 양수일때
    if (Mathf.Abs(_localVelocityX) > 2.5f || ((UseBrake || UseHandBrake) && 1<Mathf.Abs(_localVelocity)) || (UseAccele && _localVelocityZ<0) || (UseBackAccele && 0 < _localVelocityZ))
    {
        Drifting = true;            //드리프트 상태 라고 판단
        Drift();                    //드리프트 상호작용 함수 실행
    }
    else                            //그렇지 않을 시
    {
        Drifting = false;           //드리프트 상태가 아니라고 판단
        Drift();                    //드리프트 상호작용 함수 실행
    }

    AnimateWheelMeshes();//바퀴 매쉬 애니메이션 함수
}
```

1

2

3

4

휠 콜라이더를 이용한 차량 통제 시스템

위의 함수들을 이용하여 실시간으로 차량을 통제하는 Update()함수 4/4

1. 드리프트 상태를 판별하는 조건을 정리한 주석입니다.

2. 정리한 조건에 따라 드리프트 상태라고 판단되면 드리프트 상태를 활성화 하고, 드리프트의 이펙트를 담당하는 Drift();함수를 실행합니다.

3. 드리프트 상태가 아니라고 판단되면 드리프트 상태를 비활성화 하고, 드리프트의 이펙트를 담당하는 Drift();함수를 실행합니다.

4. 위에서 실행한 모든 함수의 결과로 변경되었을 휠콜라이더의 움직임을 매쉬에 적용시키는 함수인 AnimateWheelMeshes();함수를 실행합니다.

주요 구현 기술

휠 콜라이더를 이용한 차량 통제 시스템

코인의 드랍과 이동, 축적 시스템

싱글톤패턴을 이용한 차량의 업그레이드와 데이터 저장 시스템

코인의 드랍과 이동, 축적 시스템

- 드랍된 코인의 움직임을 담당하는 DropCoin.cs
- 드랍 코인의 생성 및 재활용을 관리하는 DropCoinSpawner.cs
- 차의 짐칸에 드랍코인의 충돌을 체크하는 PickUpCoinForCoinTank.cs
- 차의 짐칸에 코인을 추가하는 PlusCoin()함수
- 차의 짐칸의 코인을 빼서 옮기는 MinusCoin()함수

DropCoin.cs

```
public class DropCoin : MonoBehaviour
{

    public GameObject Target;   //드랍된 코인이 향할 타겟

    private Rigidbody _rigidbody;//드랍된 코인의 물리상태
    private PlayerController _targetPStat;//타겟이 플레이어일시 플레이어의 상태
    private bool _goCoinTank = false;//드랍된 코인의 코인탱크방향으로의 출발 상태
    private bool _goCoinInput = false;//드랍된 코인을 다른 코인 투입구로 향할때의 출발 상태
    private float _coinSpeed = 0;//드랍된코인의 이동속도
    private bool _triggerOn = true;//드랍된 코인의 트리거가 켜져있는지 상태

    private void OnEnable()
    {
        _triggerOn = true;//트리거상태 true
        gameObject.GetComponent<BoxCollider>().isTrigger = _triggerOn;//트리거 상태
박스콜리더에 적용
        _rigidbody = gameObject.GetComponent<Rigidbody>();//리지드바디 가져오기

        //출발상태 초기화
        _goCoinInput = false;
        _goCoinTank = false;

        _coinSpeed = 0;//드랍코인 이동속도
        gameObject.transform.rotation = Quaternion.identity;//드랍코인의 방향초기화
        _rigidbody.velocity = Vector3.zero;//드랍코인의 가속도 초기화
        _rigidbody.AddForce(new Vector3(Random.Range(-1f, 1f), Random.Range(10f, 15f),
Random.Range(-1f, 1f)), ForceMode.Impulse);//활성화시 위쪽방향으로 튕기기
        StartCoroutine(GoTarget());//타겟으로의 출발 시작
    }
```

1

2

코인의 드랍과 이동, 축적 시스템

드랍된 코인의 움직임을 담당하는 DropCoin.cs 1/4

1. 드랍코인의 작동에 필요한 변수들을 선언해줍니다.

2. 드랍코인이 활성화 됐을때 적용되는 OnEnable() 함수입니다.
   - 우선 드랍코인의 콜라이더의 트리거 옵션을 true로 바꿔줍니다.
   - 드랍코인의 움직임을 위해 리지드바디도 가져옵니다.
   - 이후 코인의 출발상태와 이동속도, 방향, 가속도를 초기화 해줍니다.
   - 마지막으로 드랍됐다는 강조를 위해 위쪽방향으로 한번 튕겨주고
   - 타겟으로의 출발하는 함수를 실행시켜줍니다.

DropCoin.cs

1

```
IEnumerator GoTarget()//타겟으로의 출발 함수
{
    yield return new WaitForSeconds(1f);//1초 대기
    if (Target.CompareTag("CoinTank"))//타겟이 코인탱크라면
    {
        _targetPStat = Target.GetComponent<PickUpCoinForCoinTank>().PC;//코인탱크에 지정된
플레이어의 상태 가져오기
        _goCoinTank = true;//코인탱크방향으로의 출발상태 true
    }
    else//타겟이 코인탱크가 아니라면
    {
        _goCoinInput=true;//코인투입구를 향해 출발상태 true
    }

}
```

2

```
IEnumerator UnActiveCoin()//드랍코인 활동 비활성화 함수
{
    yield return new WaitForSeconds(5f);//5초 대기
    if (!_triggerOn)//트리거가 off상태일때
    {
        gameObject.SetActive(false);//드랍코인 비활성화
    }
}
```


코인의 드랍과 이동, 축적 시스템

드랍된 코인의 움직임을 담당하는 DropCoin.cs 2/4

1. 드랍된 코인을 타겟의 방향으로 출발시키는 함수입니다.
   해당 함수가 실행되고 1초뒤
   - 드랍된 코인의 타겟이 차의 짐칸일 경우 플레이어의 상태를 가져오고, _goCoinTank (짐칸방향으로의 출발상태)를 true로 바꿔줍니다.
   - 드랍된 코인의 타겟이 차의 짐칸이 아니라면 _goCoinInput (코인투입구를 향한 출발상태)를 true로 바꿔줍니다.

2. 드랍된 코인을 비활성화 하는 함수입니다.
   해당 함수가 실행되고 5초뒤 드랍코인의 트리거 옵션이 off상태라면 드랍코인을 비활성화 해줍니다.

DropCoin.cs

```
private void FixedUpdate()
{
    if (_goCoinTank)//코인탱크 방향 출발일때
    {
        if (_triggerOn)//드랍코인의 트리거가 ON상태일때
        {
            if ((_targetPStat.Coin < _targetPStat.MaxCoin) && _targetPStat.OnStage)//플레이어의 현재 코인 갯수가 최대코인갯수보다 적고, 플레이어가 마을 밖 스테이지에 있을때
            {
                Vector3 targetPos = Target.transform.position;//타겟 위치 지정
                //타겟 방향 지정
                Vector3 dir = targetPos - transform.position;//타겟위치 - 현재 드랍코인 위치
                dir.Normalize();//방향 노멀라이즈
                transform.rotation = Quaternion.Lerp(transform.rotation, Quaternion.LookRotation(dir), _coinSpeed);//드랍코인의 방향을 타겟 방향으로 보간을 통해 부드럽게 회전
                if(_coinSpeed<100)//드랍코인속도가 100보다 작을때
                {
                    _coinSpeed += 100f * Time.deltaTime;//1초동안 100의 속도를 추가
                    if(100<_coinSpeed)//드랍코인속도가 100보다 클때
                    {
                        _coinSpeed = 100f;//드랍코인 속도는 100
                    }
                }
                _rigidbody.velocity = transform.forward * _coinSpeed;//드랍코인의 가속도 = 드랍코인의 정면쪽으로 드랍코인 속도만큼
            }
            else//플레이어의 현재 코인 갯수가 최대코인 갯수보다 많거나 같을때
            {
                _triggerOn = false;//드랍코인의 트리거상태 off
                gameObject.GetComponent<BoxCollider>().isTrigger = _triggerOn;//드랍코인의 트리거상태 적용
                _coinSpeed = 0;//드랍코인속도 0으로 초기화
                _rigidbody.velocity =Vector3.zero;//드랍코인의 가속도 0으로 초기화
                StartCoroutine(UnActiveCoin());//드랍코인의 비활성화 함수 작동
            }
        }
```

1

2

3

코인의 드랍과 이동, 축적 시스템

드랍된 코인의 움직임을 담당하는 DropCoin.cs 3/4

드랍코인이 활성화 된후 일정하게 주기적으로 실행되는 FixedUpdate()함수입니다.

1. 차의 짐칸방향으로의 출발 상태를 체크해서 true이고, 드랍코인의 트리거옵션이 true상태일때

2. 플레이어 차의 짐칸의 현재 코인 갯수가 최대코인한계 보다 작을경우 드랍코인의 방향을 차의 짐칸방향으로 회전시키면서 점점 빨라지게 가속하면서 이동시켜줍니다.

3. 플레이어의 차의 짐칸의 현재 코인 갯수가 최대코인한계 보다 많을경우 드랍코인의 트리어 옵션을 false로 바꿔주고, 드랍코인의 속도를 0으로 초기화 한 후에 일정시간후 드랍코인을 비활성화하는 함수를 작동시켜줍니다.

DropCoin.cs

1

```
        else //드랍코인의 트리거상태가 OFF일때
        {
            if(_targetPStat.Coin < _targetPStat.MaxCoin && _targetPStat.OnStage)//플레이어의
현재 코인 갯수가 최대코인갯수보다 적고, 플레이어가 마을 밖 스테이지에 있을때
            {
                _triggerOn = true;//드랍코인의 트리거상태 ON
                gameObject.GetComponent<BoxCollider>().isTrigger = _triggerOn;//드랍코인의
트리거상태 적용
                transform.LookAt(Target.transform.position);//드랍코인의 방향을 타겟쪽으로
            }
        }
    }
```

2

```
    if(_goCoinInput)//코인투입구 방향으로 출발일때
    {
        Vector3 targetPos = Target.transform.position;//타겟 위치

        //타겟 방향
        Vector3 dir = targetPos - transform.position;//타겟위지 - 드랍코인위치
        dir.Normalize();//방향 노멀라이즈

        transform.rotation = Quaternion.Lerp(transform.rotation, Quaternion.LookRotation(dir), 50
* Time.deltaTime);//드랍코인의 방향을 타겟 방향으로 보간을 통해 부드럽게 회전
        _coinSpeed += 100f * Time.deltaTime;//1초동안 100의 속도를 추가
        _rigidbody.velocity = transform.forward * _coinSpeed;//드랍코인의 가속도 = 드랍코인의
정면쪽으로 드랍코인 속도만큼
    }
  }
}
```


코인의 드랍과 이동, 축적 시스템

드랍된 코인의 움직임을 담당하는 DropCoin.cs 4/4

드랍코인이 활성화 된후 일정하게 주기적으로 실행되는 FixedUpdate()함수입니다.

1. 차의 짐칸방향으로의 출발 상태를 체크해서 true이고, 드랍코인의 트리거옵션이 false상태일때 플레이어 차의 짐칸의 현재 코인 갯수가 최대코인한계 보다 작을경우
   드랍코인의 트리거옵션을 true로 바꿔주고, 드랍코인의 방향을 차의 짐칸 방향으로 바꿔줍니다.

2. 코인투입구 방향으로의 출발상태가(_goCoinInput) true일때
   타겟의 위치를 받아와서 드랍코인의 방향을 타겟위치를 향하게 회전시키면서 가속하면서 이동시켜 줍니다.

DropCoinSpawner.cs

```
public class DropCoinSpawner : MonoBehaviour
{
  public List<GameObject> DropCoinList;//드랍코인 재활용을 위한 리스트
  public GameObject DropCoinPrefab;//드랍코인의 프리팹
  public GameObject CoinTank;//좀비에게서 드랍된 코인을 넣을 코인탱크
  public GameObject CoinHouse;//코인탱크에서 드랍된 코인을 넣을 코인하우스
```

1

```
  public void CoinDrop(Vector3 pos)//좀비에게서 코인을 드랍하는 함수 (pos = 죽은 좀비의 위치)
  {
    bool succes = false;//재활용 성공 상태
    for (int i = 0; i < DropCoinList.Count; i++)//재활용 리스트만큼 반복
    {
      if (!(DropCoinList[i].gameObject.activeSelf))//i번째 리스트의 오브젝트(드랍코인)이 비활성화 상태일때
      {
        DropCoinList[i].transform.position = pos;//i번째 리스트의 드랍코인 위치를 pos(죽은 좀비의 위치)로 이동
        DropCoinList[i].GetComponent<DropCoin>().Target = CoinTank;//드랍코인의 타겟을 코인탱크로 지정
        DropCoinList[i].SetActive(true);//드랍코인을 활성화
        succes = true;//재활용 성공
        break;//반복 종료
      }
    }
    if (!succes)//재활용 실패시
    {
      DropCoinList.Add(Instantiate(DropCoinPrefab, pos, Quaternion.identity));//드랍코인을 pos(죽은 좀비의 위치)에 소환
      DropCoinList[DropCoinList.Count - 1].GetComponent<DropCoin>().Target= CoinTank;//생성된 드랍코인의 타겟을 코인탱크로 지정
      DropCoinList[DropCoinList.Count - 1].transform.parent = transform;//드랍코인스포너를 생성된 드랍코인의 부모오브젝트로 지정
    }
  }
```

2

코인의 드랍과 이동, 축적 시스템

드랍 코인의 생성 및 재활용을 관리하는 DropCoinSpawner.cs 1/2

1. 드랍코인의 생성과 재활용에 필요한 변수들을 선언해줍니다.

2. 좀비를 죽였을때 코인을 드랍하는 CoinDrop()함수입니다.
   - 우선 재활용 성공 상태를 체크할 bool변수 succes를 선언해줍니다.
   - 드랍코인의 재활용리스트를 맨 앞부터 확인하여 한번이라도 비활성화 상태의 드랍코인이 있을시
     해당 드랍코인을 좀비 위치로 이동합니다.
     차의 짐칸을 타겟으로 지정해주면서 활성화합니다.
     재활용 성공 상태를 true로 변경합니다.
     그리고 반복을 종료합니다.
   - for문 종료 후 재활용 성공 상태가 false 이면
     좀비가 죽은 위치에 드랍코인을 소환해줍니다.
     이후 생성된 드랍코인의 타겟을 차의 짐칸으로 지정해줍니다.
     마지막으로 생성된 드랍코인 스포너를 생성된 드랍코인의 부모로 지정해줍니다.

DropCoinSpawner.cs

```
public void CoinInput(Vector3 pos)
{
    bool succes = false;//재활용 성공 상태
    for (int i = 0; i < DropCoinList.Count; i++)//재활용 리스트만큼 반복
    {
        if (!(DropCoinList[i].gameObject.activeSelf))//i번째 리스트의 오브젝트(드랍코인)이 비활성화 상태일때
        {
            DropCoinList[i].transform.position = pos;//i번째 리스트의 드랍코인 위치를 pos(코인탱크 내부의 코인위치)로 이동
            DropCoinList[i].GetComponent<DropCoin>().Target = CoinHouse;//드랍코인의 타겟을 코인하우스로 지정
            DropCoinList[i].SetActive(true);//드랍코인을 활성화
            succes = true;//재활용 성공
            break;//반복 종료
        }
    }
    if (!succes)//재활용 실패시
    {
        DropCoinList.Add(Instantiate(DropCoinPrefab, pos, Quaternion.identity));//드랍코인을 pos(코인탱크 내부의 코인위치)에 소환
        DropCoinList[DropCoinList.Count - 1].GetComponent<DropCoin>().Target = CoinHouse;//생성된 드랍코인의 타겟을 코인하우스로 지정
        DropCoinList[DropCoinList.Count - 1].transform.parent = transform;//드랍코인스포너를 생성된 드랍코인의 부모오브젝트로 지정
    }
}
```

1

2

3

코인의 드랍과 이동, 축적 시스템

드랍 코인의 생성 및 재활용을 관리하는 DropCoinSpawner.cs 2/2

차의 짐칸에서 마을의 코인투입구로 코인을 전달하는 CoinInput()함수입니다.

1. 우선 재활용 성공 상태를 체크할 bool변수 succes를 선언해줍니다.
2. 드랍코인의 재활용리스트를 맨 앞부터 확인하여 한번이라도 비활성화 상태의 드랍코인이 있을시
   - 해당 드랍코인을 짐칸안의 코인위치로 이동시켜줍니다.
   - 마을의 코인투입구로 타겟으로 지정해주면서 활성화합니다.
   - 재활용 성공 상태를 true로 변경합니다.
   - 그리고 반복을 종료합니다.
3. for문 종료 후 재활용 성공 상태가 false 이면
   - 짐칸안의 코인 위치에 드랍코인을 소환해줍니다.
   - 이후 생성된 드랍코인의 타겟을 마을의 코인투입구로 지정해줍니다.
   - 마지막으로 생성된 드랍코인 스포너를 생성된 드랍코인의 부모로 지정해줍니다.

PickUpCoinForCoinTank.cs

```
public class PickUpCoinForCoinTank : MonoBehaviour
{
  public GameObject Player;//플레이어 게임 오브젝트
  public PlayerController PC;//플레이어의 상태를 관리하는 스크립트
  public CoinSounds CS;//코인사운드 관리하는 스크립트
  private int Coin;//현재 코인갯수
  private int MaxCoin;//코인탱크가 가질수 있는 최대 코인 갯수
```

1

```
  private void Start()
  {
    PC = Player.GetComponent<PlayerController>();
  }
```

2

```
  private void OnTriggerEnter(Collider other)
  {
    if(other.CompareTag("DropCoin"))//좀비로부터 드랍된 코인의 코인탱크 트리거시
    {

      Coin = PC.Coin;//플레이어 차의 상태에서 현재 코인 갯수를 받아옴
      MaxCoin = PC.MaxCoin;//플레이어 차의 상태에서 최대코인 갯수를 받아옴
      if (Coin < MaxCoin)//최대 코인 갯수보다 현재 코인 갯수가 적을때
      {
        if(other.GetComponent<DropCoin>().Target.CompareTag("CoinTank"))//드랍코인의
목표가 코인탱크일때
        {
          CS.OnCoinSound(other.transform.position);//코인사운드 출력
          other.gameObject.SetActive(false);//드랍코인비활성화
          PC.PlusCoin(other.transform.position);//플레이어상태에서 코인추가,짐칸에
코인소환또는 활성화
        }
      }
    }
  }
}
```

3

코인의 드랍과 이동, 축적 시스템

차의 짐칸에 드랍코인의 충돌을 체크하는 PickUpCoinForCoinTank.cs

1. 짐칸과 드랍코인의 충돌체크 후 작동에 필요한 변수들을 선언해줍니다.

2. 게임이 시작되면 PC변수에 플레이어의 상태를 불러옵니다.

3. 차의 짐칸에 다른 오브젝트가 트리거안으로 들어왔을때 호출되는 OnTriggerEnter()함수입니다.
   - 만약 트리거된 오브젝트의 태그가 “DropCoin”(드랍코인)이라면
   - PC변수에서 현재 짐칸에 실려있는 코인의 갯수를 받아오고,
   - 짐칸에 실을수 있는 최대코인의 갯수를 받아옵니다.
   - 만약 최대 코인 갯수보다 현재 코인갯수가 적으면서 드랍코인의 목표가 차의 짐칸일 경우
   - 코인의 사운드를 출력해주고, 충돌한 드랍코인을 비활성화 해주면서
   - 플레이어의 상태에서 짐칸의 코인 갯수를 늘려주고 늘어난 만큼 짐칸에 코인을 소환하거나 재활용해서 활성화 시켜줍니다.

PlayerController.cs

```
public void PlusCoin(Vector3 pos)
  {
    bool succes = false;                                                    1
    for (int i = 0; i < CoinList.Count; i++)
    {
      if (!(CoinList[i].gameObject.activeSelf))                             2
      {
        CoinList[i].transform.position = pos;
        CoinList[i].SetActive(true);
        succes = true;
        break;
      }
    }
    if (!succes)
    {
      CoinList.Add(Instantiate(CoinPrefab, pos, Quaternion.identity));      3
      CoinList[CoinList.Count - 1].transform.parent = CoinTank.transform;
    }
    Coin++;
    CoinText.text = Coin + " / " + MaxCoin;                                 4
    CoinImage.fillAmount = (float)Coin / MaxCoin;
  }
```


코인의 드랍과 이동, 축적 시스템

차의 짐칸에 코인을 추가하는 PlusCoin()함수

1. 우선 재활용 성공 상태를 체크할 bool변수 succes를 선언해줍니다.

2. 코인의 재활용리스트를 맨 앞부터 확인하여 한번이라도 비활성화 상태의 코인이 있을시
   - 해당 코인을 짐칸에 충돌한 드랍코인의 위치로 이동시켜줍니다.
   - 이동된 코인을 활성화합니다.
   - 재활용 성공 상태를 true로 변경합니다.
   - 그리고 반복을 종료합니다.

3. for문 종료 후 재활용 성공 상태가 false 이면
   - 짐칸에 충돌한 드랍코인의 위치에 코인을 소환해줍니다.
   - 차의 짐칸을 생성된 코인의 부모로 지정해줍니다.

4. 차의 상태에서 짐칸에 실린 코인의 갯수를 1 더해주고, UI에서 표시해줍니다.

PlayerController.cs

```
void MinusCoin()
{
    for(int i=0;i<CoinList.Count;i++)
    {
        if(CoinList[i].gameObject.activeSelf)
        {

DropCoinSpawner.GetComponent<DropCoinSpawner>().CoinInput(CoinList[i].transform.position);
            CoinList[i].SetActive(false);
            break;
        }
    }
    Coin--;
    CoinText.text = Coin + " / " + MaxCoin;
    CoinImage.fillAmount = (float)Coin / MaxCoin;
}
```

1

2

코인의 드랍과 이동, 축적 시스템

차의 짐칸의 코인을 빼서 옮기는 MinusCoin()함수

1. 코인의 재활용리스트를 맨 앞부터 확인하여 한번이라도 활성화 상태의 코인이 있을시
   - 드랍코인스포너에서 코인위치에서 코인투입구를 타겟으로 하는 드랍코인을 소환또는 재활성화해주고, 해당 코인의 위치를 전달해줍니다.
   - 해당 코인을 비활성화합니다.
   - 그리고 반복을 종료합니다.

2. 차의 상태에서 짐칸에 실린 코인의 갯수를 1 빼주고, UI에서 표시해줍니다.

주요 구현 기술

휠 콜라이더를 이용한 차량 통제 시스템

코인의 드랍과 이동, 축적 시스템

싱글톤패턴을 이용한 차량의 업그레이드와 데이터 저장 시스템

싱글톤 패턴을 이용한 차량의 업그레이드와 데이터 저장 시스템

- 싱글톤 패턴을 이용한 차량 업그레이드 상태를 저장하고 불러오는 StatusManager.cs
- 스테이터스매니저에 저장된 업그레이드 상태를 UI로 불러와 조작하게 해주는 Upgrade.cs

StatusManager.cs

```
public class StatusManager : MonoBehaviour
{
    #region Singleton
    private static StatusManager s_instance = null;
    public static StatusManager Instance
    {
        get
        {
            if (s_instance == null) return null;
            return s_instance;
        }
    }
    private void Awake()
    {
        if(s_instance == null)
        {
            s_instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else
        {
            Destroy(gameObject);
        }
    }
    #endregion
```


싱글톤 패턴을 이용한 차량의 업그레이드와 데이터 저장 시스템

싱글톤 패턴을 이용한 차량 업그레이드 상태를 저장하고 불러오는 StatusManager.cs 1/4

싱글톤 패턴 구현 코드입니다.

StatusManager.cs

1

```
public int WeaponNum=0;
public int WeaponDamage=0;
public int MaxSpeed = 0;
public int MaxAccele = 0;
public int MaxHP = 0;
public int HPRegen = 0;
public int CoinTank = 0;
public int WorldCoin = 0;
public int NextUpgradePrice = 10;
public int Score=0;
public int SaveNum = 0; //세이브데이터 넘버
public List<int> StageOpen; //스테이지 오픈 상태
public int MaxStage = 2;  //최대 스테이지
```

1

```
private void Start()
{

    StageOpen.Clear();
    for (int i = 0; i < MaxStage; i++)
    {
        StageOpen.Add(0);
    }
}
```


싱글톤 패턴을 이용한 차량의 업그레이드와 데이터 저장 시스템

싱글톤 패턴을 이용한 차량 업그레이드 상태를 저장하고 불러오는 StatusManager.cs 2/4

1. 차량의 업그레이드 상태와 스테이지 정보, 세이브 데이터 넘버를 저장할 변수를 선언해줍니다.

2. 시작시 스테이지 오픈 상태 리스트<정수> 를 초기화 해줍니다.

StatusManager.cs

1

```
public void LoadGame()
{
    WeaponNum    = PlayerPrefs.GetInt("WeaponNum" + SaveNum, 0);
    WeaponDamage = PlayerPrefs.GetInt("WeaponDamage" + SaveNum, 0);
    MaxSpeed     = PlayerPrefs.GetInt("MaxSpeed" + SaveNum, 0);
    MaxAccele    = PlayerPrefs.GetInt("MaxAccele" + SaveNum, 0);
    MaxHP        = PlayerPrefs.GetInt("MaxHP" + SaveNum, 0);
    HPRegen      = PlayerPrefs.GetInt("HPRegen" + SaveNum, 0);
    CoinTank     = PlayerPrefs.GetInt("CoinTank" + SaveNum, 0);
    WorldCoin    = PlayerPrefs.GetInt("WorldCoin" + SaveNum, 0);
    NextUpgradePrice = PlayerPrefs.GetInt("NextUpgradePrice" + SaveNum, 0);
    Score = PlayerPrefs.GetInt("Score" + SaveNum, 0);
    for (int i = 0; i < MaxStage; i++)
    {
        StageOpen[i]=PlayerPrefs.GetInt("StageOpen" + i + SaveNum, 0);
    }
}
```

2

```
public void SaveGame()
{
    PlayerPrefs.SetInt("WeaponNum" + SaveNum, WeaponNum);
    PlayerPrefs.SetInt("WeaponDamage" + SaveNum, WeaponDamage);
    PlayerPrefs.SetInt("MaxSpeed" + SaveNum, MaxSpeed);
    PlayerPrefs.SetInt("MaxAccele" + SaveNum, MaxAccele);
    PlayerPrefs.SetInt("MaxHP" + SaveNum, MaxHP);
    PlayerPrefs.SetInt("HPRegen" + SaveNum, HPRegen);
    PlayerPrefs.SetInt("CoinTank" + SaveNum, CoinTank);
    PlayerPrefs.SetInt("WorldCoin" + SaveNum, WorldCoin);
    PlayerPrefs.SetInt("Score" + SaveNum, Score);
    PlayerPrefs.SetInt("NextUpgradePrice" + SaveNum, NextUpgradePrice);
    for (int i = 0; i < MaxStage; i++)
    {
        PlayerPrefs.SetInt("StageOpen"+i + SaveNum, StageOpen[i]);
    }
}
```


싱글톤 패턴을 이용한 차량의 업그레이드와 데이터 저장 시스템

싱글톤 패턴을 이용한 차량 업그레이드 상태를 저장하고 불러오는 StatusManager.cs 3/4

1. 플레이어프렙을 사용하여 지정된 세이브넘버로 저장되어있는 세이브 데이터의 업그레이드 상태와 스테이지 상태를 불러와서 현재 스테이터스에 적용하는 LoadGame()함수입니다.

2. 플레이어프렙을 사용하여 현재 스테이터스의 업그레이드 상태와 스테이지 상태를 지정된 세이브넘버의 세이브데이터에 저장하는 SaveGame()함수입니다.

StatusManager.cs

```
public void NewGame()
{
    WeaponNum    =    0;
    WeaponDamage =    0;
    MaxSpeed     =    0;
    MaxAccele    =    0;
    MaxHP        =    0;
    HPRegen      =    0;
    CoinTank     =    0;
    WorldCoin    =    0;
    Score =           0;
    NextUpgradePrice = 10;
    for(int i=0;i<MaxStage; i++)
    {
        StageOpen[i] = 0;
    }
    SaveGame();
}

}
```

1

싱글톤 패턴을 이용한 차량의 업그레이드와 데이터 저장 시스템

싱글톤 패턴을 이용한 차량 업그레이드 상태를 저장하고 불러오는 StatusManager.cs 4/4

1. 지정된 세이브넘버의 세이브데이터를 초기화 해주는 NewGame()함수입니다.
   - 모든 변수들을 초기화해줍니다.
   - SaveGame()함수를 호출하여 지정된 세이브넘버에 저장해줍니다.

Upgrade.cs

```
public class Upgrade : MonoBehaviour
{
    public List<Image> Weapon;
    public TMP_Text Damage;
    public TMP_Text MaxSpeed;
    public TMP_Text MaxAccele;
    public TMP_Text MaxHP;
    public TMP_Text HPRegen;
    public TMP_Text CoinTank;
    public TMP_Text NextPrice;
    public GameObject Player;
    private PlayerController   PC;
```

1

```
    private void OnEnable()
    {
        _PC = Player.GetComponent<PlayerController>();
        for(int i=0;i<3;i++)
        {
            if(StatusManager.Instance.WeaponNum ==i)
            {
                Weapon[i].gameObject.SetActive(true);
            }
            else
            {
                Weapon[i].gameObject.SetActive(false);
            }
        }

        Damage.text = StatusManager.Instance.WeaponDamage+"";
        MaxSpeed.text = StatusManager.Instance.MaxSpeed+"";
        MaxAccele.text = StatusManager.Instance.MaxAccele + "";
        MaxHP.text = StatusManager.Instance.MaxHP + "";
        HPRegen.text = StatusManager.Instance.HPRegen + "";
        CoinTank.text = StatusManager.Instance.CoinTank + "";
        NextPrice.text = StatusManager.Instance.NextUpgradePrice + "";
    }
```

2

싱글톤 패턴을 이용한 차량의 업그레이드와 데이터 저장 시스템

스테이터스매니저에 저장된 업그레이드 상태를 UI로 불러와 조작하게 해주는 Upgrade.cs 1/3

1. 업그레이드상태를 표시할 UI들과 플레이어의 오브젝트, 플레이어의 상태를 받아올 변수를 선언해줍니다.

2. 해당 스크립트가 적용된 업그레이드UI창이 활성화 되었을때
   - _PC 변수에 플레이어의 상태를 불러오고,
   - 싱글톤패턴이 적용된 스테이터스 매니저에서 업그레이드 상태를 불러와서 UI에 표시합니다.
   - 무기종류에 관련된 업그레이드상태는 적용된 무기의 종류에 따라 이미지를 다르게 표시합니다.

Upgrade.cs

```
public void WeaponButton()
{
    switch(StatusManager.Instance.WeaponNum)
    {
        case 0:
            if (5 <= StatusManager.Instance.WeaponDamage)
            {
                StatusManager.Instance.WeaponNum++;
                _PC.WeaponChange();
            }
            break;
        case 1:
            if (10 <= StatusManager.Instance.WeaponDamage)
            {
                StatusManager.Instance.WeaponNum++;
                _PC.WeaponChange();
            }
            break;
        case 2:
            StatusManager.Instance.WeaponNum = 0;
            _PC.WeaponChange();
            break;
    }
    for (int i = 0; i < 3; i++)
    {
        if (StatusManager.Instance.WeaponNum == i)
        {
            Weapon[i].gameObject.SetActive(true);
        }
        else
        {
            Weapon[i].gameObject.SetActive(false);
        }
    }

}
```

1

2

싱글톤 패턴을 이용한 차량의 업그레이드와 데이터 저장 시스템

스테이터스매니저에 저장된 업그레이드 상태를 UI로 불러와 조작하게 해주는 Upgrade.cs 2/3

무기 종류를 업그레이드하는 버튼의 WeaponButton()함수입니다.

버튼이 눌렸을때

1. 스위치 문으로 스테이터스매니저의 무기번호를 받아옵니다.
    - 무기번호가 0일때 스테이터스매니저의 무기데미지가 5 이상이면
    - 스테이터스매니저의 무기번호를 1 올리고, 플레이어의 무기를 바꿉니다.
    - 무기번호가 1일때 스테이터스매니저의 무기데미지가 10 이상이면
    - 스테이터스매니저의 무기번호를 1 올리고, 플레이어의 무기를 바꿉니다.
    - 무기번호가 2일때는 무기번호를 0으로 바꾸고 플레이어의 무기를 바꿉니다.
2. 바뀐 무기 번호에 맞게 무기 종류 UI의 이미지를 바꿔줍니다.

Upgrade.cs

```
    public void DamageButton()
    {
        if (StatusManager.Instance.NextUpgradePrice <= StatusManager.Instance.WorldCoin)
        {
            StatusManager.Instance.WorldCoin = StatusManager.Instance.WorldCoin -
StatusManager.Instance.NextUpgradePrice;
            StatusManager.Instance.WeaponDamage++;
            StatusManager.Instance.NextUpgradePrice++;
            Damage.text = StatusManager.Instance.WeaponDamage + "";
            NextPrice.text = StatusManager.Instance.NextUpgradePrice + "";
            _PC.PlusDamage();
        }
    }

    public void MaxSpeedButton()
    {
        if (StatusManager.Instance.NextUpgradePrice <= StatusManager.Instance.WorldCoin)
        {
            StatusManager.Instance.WorldCoin = StatusManager.Instance.WorldCoin -
StatusManager.Instance.NextUpgradePrice;
            StatusManager.Instance.MaxSpeed++;
            StatusManager.Instance.NextUpgradePrice++;
            MaxSpeed.text = StatusManager.Instance.MaxSpeed + "";
            NextPrice.text = StatusManager.Instance.NextUpgradePrice + "";
            _PC.PluseMaxSpeed();
        }
    }

    public void ExitButton()
    {
        gameObject.SetActive(false);
    }
}
```


싱글톤 패턴을 이용한 차량의 업그레이드와 데이터 저장 시스템

스테이터스매니저에 저장된 업그레이드 상태를 UI로 불러와 조작하게 해주는 Upgrade.cs 3/3

1. 나머지 업그레이드 스탯의 버튼에 적용되는 함수들입니다.
   종류별로 DamageButton(), MaxSpeedButton(), MaxAcceleButton(), MaxHPButton(), HPRegenButton(), CoinTankButton() 등이 있고, 형식은 동일합니다.
   - 버튼을 클릭시 스테이터스 매니저에서 불러온 업그레이드에 필요한 가격이 가지고 있는 월드코인보다 적을경우
   - 클릭한 업그레이드 스탯의 수치를 1 올립니다.
   - 다음 업그레이드에 필요한 가격의 수치도 1 올립니다.
   - 변경된 수치를 UI에 표시합니다.
   - 변경된 수치를 플레이어의 상태에 적용합니다.

2. 업그레이드 UI 창을 닫는 버튼 ExitButton()함수입니다.
   클릭시 업그레이드 UI창을 비활성화 합니다.

Thank You!

감사합니다!